

PREFACE FOR FACULTY

When the sequence length N is an integral power of 4 ($N = 4^M$), the Radix-4 FFT algorithm provides an additional 25% reduction in complex multiplications compared to Radix-2 by processing 4 points per butterfly.

Pedagogical Strategy:

- Derive the Radix-4 decimation equation by breaking the N -point sum into 4 sub-sequences.
- Formulate the 4-point DFT butterfly matrix kernel:
$$\mathbf{W}_4 = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & -j & -1 & j \\ 1 & -1 & 1 & -j \\ 1 & j & -1 & -j \end{bmatrix}$$
- Highlight that multiplications by ± 1 and $\pm j$ are trivial (cost zero actual hardware multipliers).
- Prove that a Radix-4 butterfly requires only **3 complex twiddle multiplications** (compared to 8 in an equivalent 2-stage Radix-2 block).
- Compare total complex multiplication count: $\mu_{\text{Radix-4}} = \frac{3N}{8} \log_2 N = \frac{3N}{4} \log_4 N$.

1. LEARNING OBJECTIVES

By the end of this lecture, students will be able to:

- Derive** the Radix-4 FFT decimation equations and 4-point butterfly structure.
- Construct** digit-reversal (base-4) addressing schemes for Radix-4 input/output ordering.
- Quantify** arithmetic operation counts across Direct DFT, Radix-2, and Radix-4 algorithms.
- Evaluate** trade-offs between algorithm speed and hardware architectural complexity.

2. MATHEMATICAL FOUNDATIONS

2.1 Radix-4 Decimation-in-Time Derivation

Let $N = 4^M$. Decimate $x[n]$ into 4 sub-sequences: $x[4r], x[4r+1], x[4r+2], x[4r+3]$ for $r = 0, 1, \dots, N/4 - 1$:

$$X[k] = \sum_{r=0}^{N/4-1} x[4r] W_N^{4rk} + W_N^k \sum_{r=0}^{N/4-1} x[4r+1] W_N^{4rk} + W_N^{2k} \sum_{r=0}^{N/4-1} x[4r+2] W_N^{4rk} + W_N^{3k} \sum_{r=0}^{N/4-1} x[4r+3] W_N^{4rk}$$

Using $W_N^{4rk} = W_{N/4}^{rk}$:

$$X[k] = X_0[k] + W_N^k X_1[k] + W_N^{2k} X_2[k] + W_N^{3k} X_3[k]$$

Evaluating for $k, k + N/4, k + N/2, k + 3N/4$ yields the Radix-4 Butterfly Matrix:

$$\begin{bmatrix} X[k] & X[k + N/4] & X[k + N/2] & X[k + 3N/4] \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & -j & -1 & j \\ 1 & -1 & 1 & -j \\ 1 & j & -1 & -j \end{bmatrix} \begin{bmatrix} X_0[k] & X_1[k] W_N^k & X_2[k] W_N^{2k} & X_3[k] W_N^{3k} \end{bmatrix}$$

2.2 Computational Complexity Comparison

Algorithm	Complex Multiplications	Complex Additions	For $N = 1024$ Mults
Direct DFT	N^2	$N(N-1)$	1,048,576
Radix-2 FFT	$\frac{N}{2} \log_2 N$	$N \log_2 N$	5,120
Radix-4 FFT	$\frac{3N}{8} \log_2 N = \frac{3N}{4} \log_4 N$	$\frac{N}{2} (4 \log_4 N) = 2N \log_4 N$	3,840

3. WORKED NUMERICAL EXAMPLES

Example 12.1: Arithmetic Complexity Evaluation

Problem: A radar DSP processor must compute a 4096-point DFT in real-time. Calculate the complex multiplications and additions for:
(a) Direct DFT, (b) Radix-2 FFT, (c) Radix-4 FFT.

Solution: Here $N = 4096 = 2^{12} = 4^6$.

- **(a) Direct DFT:**
 - Multiplications: $N^2 = 4096^2 = 16,777,216$.
 - Additions: $N(N-1) = 4096 \times 4095 = 16,773,120$.
- **(b) Radix-2 FFT:**
 - Multiplications: $\frac{N}{2} \log_2 N = 2048 \times 12 = 24,576$.
 - Additions: $N \log_2 N = 4096 \times 12 = 49,152$.
- **(c) Radix-4 FFT:**
 - Multiplications: $\frac{3N}{8} \log_2 N = \frac{3 \times 4096}{8} \times 12 = 1,536 \times 12 = 18,432$.
 - Additions: $N \log_2 N \times \text{matrix factor} \approx 49,152$.
- **Savings:** Radix-4 achieves a $(24576 - 18432)/24576 = 25.0$ reduction in complex multiplications compared to Radix-2.

4. UNIVERSITY EXAMINATION QUESTIONS & MARKING RUBRIC

Question 1 (15 Marks)

(a) Derive the Radix-4 Decimation-in-Time FFT algorithm. Explain why it is computationally more efficient than the Radix-2 FFT. *(10 Marks)* **(b)** Construct the Digit-Reversal mapping table for a 16-point Radix-4 FFT. *(5 Marks)*

Model Answer & Step-by-Step Marking Rubric:

- **Part (a):**
 - 4-way decimation derivation and matrix equation *(5 Marks)*
 - Trivial twiddle multiplication proof $(\pm 1, \pm j)$ requiring only 3 complex twiddles per 4-point block *(3 Marks)*
 - Derivation of $\mu = \frac{3N}{8} \log_2 N$ showing 25% savings over Radix-2 *(2 Marks)*
- **Part (b):**
 - 16-point base-4 digit reversal $(n = d_1 4^1 + d_0 4^0 \leftrightarrow n_{\text{rev}} = d_0 4^1 + d_1 4^0)$ table for $n = 0$ to 15 *(5 Marks)*

5. PYTHON VERIFICATION SCRIPT

```
N = 4096
r2_mults = (N // 2) * 12
r4_mults = (3 * N // 8) * 12
print(f"Radix-2 Mults: {r2_mults:,}")
print(f"Radix-4 Mults: {r4_mults:,}")
print(f"Percentage Savings: {100*(r2_mults - r4_mults)/r2_mults:.2f}%")
```